[Interview Question Breakdown] Given 10 HTTP API requests within 5 seconds, explain your thought process using data structures and algorithms.

Original Published on 2026-03-20 15:00:00 · 335 reads · 9 · 3 · Copyright 4.0 BY-SA

Article tags: #interview #http #Data Structures

Eight-legged essay This article is included in the column. 39 articles Subscribe to column

New Star Cup 14-Day Creative Challenge Camp - 20th Session 100,000+ views 420 people participated

The interviewer asked:

> When developing a web project, some HTTP interfaces are exposed. I want to rate limit these interfaces, restricting access to only 10 times within 5 seconds. Using data structures and algorithms, how can this be implemented? Please explain your thought process.

They only mentioned a sliding window and nothing else, and that was it...

First, why limit traffic?

There are three main reasons:

1. To prevent malicious API attacks, such as web crawlers scraping data or DDoS attacks by hackers.
2. Protect the server from being overwhelmed, for example, if one interface goes down, it could bring down the entire service.
3. To ensure the availability of core business processes, for example, during peak sales periods, payment interfaces should be prioritized while rate limiting should be applied to regular interfaces.

Technology Selection

plan	Data Structures + Algorithms	Core idea	Advantages and disadvantages	Applicable Scenarios
Fixed window counter	Array/Map + timestamp	Divide the time into fixed windows, and count within each window.	It is simple to implement; however, it has **a criticality issue** (for example, it refreshes 10 times in 4.9 seconds, refreshes another 10 times in 5.1 seconds, and actually refreshes 20 times in 0.2 seconds).	For scenarios where high precision is not required
Sliding window counter	Local: LinkedList (doubly linked list) Distributed: Redis ZSet (sorted set) + timestamp	The window is "sliding," only retaining requests within the currently active window.	Solved the criticality problem of fixed windows; the local version of LinkedList has O(1) time complexity for deleting the head and adding the tail, while the distributed version of ZSet is efficient for time range operations; the local version is not suitable for distributed clusters.	Scenarios requiring precise counting within a fixed time window
Leaky Bucket Algorithm	Queue + Timer	Like a leaky bucket, requests are first stored in a queue and then processed at a constant rate.	It can smooth out peak and off-peak traffic; however, it cannot handle sudden surges in traffic (many requests are rejected during flash sales).	Scenarios requiring high traffic stability (such as message queues)
Token Bucket Algorithm	Queue + Timer	Like a token bucket, tokens are generated at a constant rate, and a request can only pass if it obtains a token.	It can smooth out peak traffic and fill valleys, and also handle sudden traffic surges.	Scenarios requiring both traffic stability and protection against sudden traffic spikes (such as API gateways).

Sliding window counter

The core of a sliding window is that the window is no longer static and segmented, but moves continuously and overlaps with each other over time:

For example, a fixed window is static; its boundaries remain unchanged for 0-5 seconds or 5

It's pretty good, isn't it? One-click collection

CodeEdge_Xu focus on 9 3 0 share tip ••• Column Directory

Data Structure Selection

Local: LinkedList (doubly linked list)

We chose a doubly linked list because we need to frequently delete the expired timestamp at the head of the list and frequently add new request timestamps at the tail of the list. The time complexity of deleting the head and adding the tail operations of a doubly linked list is O(1), which is extremely efficient.

Distributed: Redis ZSet (Sorted Set)

Web projects are mostly deployed in clusters, and local counters cannot be synchronized across multiple instances, so Redis is considered as a solution.

Choose the ZSet data structure in Redis. ZSet `score` fields can store the timestamp of the request and natively support sorting by time range.

Implementation logic

Each incoming request executes 3 steps:

1. Delete all requests earlier than the current time - window size.
2. Count the number of requests in the current window; reject requests exceeding 10.
3. Add the timestamp of the current request to the data structure and allow the request.

Tips

1. The token generation rate of the token bucket should be reasonable.

It can't be too fast (which is equivalent to no rate limiting) nor too slow (which would result in many requests being rejected). It needs to be adjusted according to the business scenario.

2. The expiration time of Redis ZSet needs to be set correctly.

The window size should be set plus a small buffer (eg, 1 second) to prevent data from expiring prematurely and causing inaccurate counting.

去阿里云养一只龙虾，省下一台 MacMini 广告
阿里云轻量应用服务器或无影云电脑上，一键拉起openclaw镜像，直接通过iMessage或钉钉与你的Bot互动

[Compilation of Java Interview Questions] MySQL Database Section (2026 Edition) The best time to plant a tree was ten years ago, the… 9252
Clustered and non-clustered indexes, underlying data structures of indexes , B-trees and B+ trees, why MySQL uses B+ trees instead of red-black trees, database engines, InnoDB…

[2025 Huawei OD Exam Questions] - Smart Score Sheet (C&D Version - 100 points) (JS Version) **My Coding Family** 1030
🔔This article is included in the "Latest and Most Complete Huawei OD Exam Questions (JavaScript Version)" column, providing a step-by-step guide for beginners to pass the Hu…

Xiaohongshu Interview Questions - Interview Framework and Interview Skills Summary daguan2012's Blog 960
This article systematically outlines the core framework and techniques for interviewing Java development engineers at Xiaohongshu , focusing on three key dimensions: 1) Intervie…

JAVA Data Structures DAY6 - Stacks and Queues HRDMN's Blog 837
This article is part of a Java data structure learning notes series, focusing on two fundamental data structures : stacks and queues . It details the concepts, core methods, simulatio…

Sweeping the Golden March and Silver April of 2026: In-depth analysis of Java backend interview "formulaic questions" and detailed explana… DZ Space 545
2026 Java Backend Interview Trends and Preparation Guide: The 2026 Java backend interview landscape shows a significant trend of differentiation: competition for entry-level pos…

Interviewer: How do you set the rate limiting threshold for your system ? IT Yang Xiucai's Blog 1263
How to scientifically set rate limiting thresholds? This article systematically reviews the core elements of rate limiting technology, from static/dynamic algorithm comparisons to clust…

Watch animations, get offers: Comprehensive analysis of real algorithm interview questions from top companies GitChat 1604
This comprehensive video course includes 6.5 hours of optional videos, 36 accompanying text and image explanations, 20 coding practice problems, and complete permanent acce…

After six interviews in half a month , I finally got to the HR interview at Baidu. Wang Zhongyang talks about programming 2178
Today I 'd like to share a post from a fan who went on a crazy interview spree in the last two weeks. He interviewed for two positions at Baidu alone, and had a total of six interviews…

The 5th Edition of "The Programmer's Interview Bible
First, at the level of ** data structures and algorithms **, this

It's pretty good, isn't it? One-click collection

CodeEdge_Xu focus on 9 3 0 share tip ••• Column Directory